FinNova Bank

Android App Guide · Java · XML · MVP

What's inside

1. How the app works, screen by screen
2. MVP architecture and the project structure
3. Every validation rule on the Profile pages
4. Task 1: Profile page validation, step by step in Android Studio
5. Testing, debugging, Git workflow and a practice backlog

Prepared for Eswar Reddy Madhira · October 2026

Repository: github.com/iEswar23/FinNova-Bank

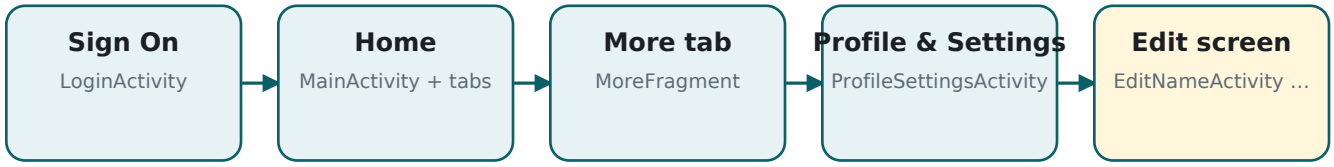
Practice project with a fictional brand and a fake in-memory backend.

1. What you are building

FinNova Bank is a practice Android banking app written in **Java** with **XML layouts** and the **MVP (Model–View–Presenter)** pattern, the same stack many large bank codebases still use. Its centre is a **Profile & Settings** screen modelled on the kind of screen you work on day to day: a Profile section (preferred first name, address, email, mobile phone, income) and a Settings section (User ID and password, login options, alerts and more). Each profile item opens an edit screen with complete validation.

Everything runs on the device. A **fake backend** (`FakeBankRepository`) answers after a short delay and returns real-looking server errors, so you can practise the full flow: type → validate → call the server → handle success or error → refresh the previous screen.

Screen flow



Save → finish() → onResume() of Profile & Settings reloads the profile, so the new value shows up.

Feature list

Screen	What it shows / does	Main files
Sign On	User ID + password, required checks, wrong-credentials dialog	LoginActivity, LoginPresenter
Accounts tab	Greeting and 3 accounts (checking, savings, card) in a RecyclerView	AccountsFragment, AccountsPresenter, AccountsAdapter
Benefits / Offers tabs	Static content cards	InfoFragment
More tab	Profile & Settings, Help, Sign Off (confirm dialog)	MoreFragment
Profile & Settings	Two card sections with 11 rows; current values shown, email and phone masked	ProfileSettingsActivity + Presenter + Contract
Preferred first name	Letters with single space / - / ' between, max 30, must differ from current	profile/name/*
Email address	Format, confirm-email match, server error "already in use"	profile/email/*
Mobile phone	US rules, formats while typing, sends 10 digits	profile/phone/*
Address	No P.O. box, state dropdown, ZIP or ZIP+4, focus jumps to first error	profile/address/*
Income information	Employment status, annual income, monthly housing (cross-field rule)	profile/income/*
User ID and Password	Live password checklist, 9 password rules, 2 server errors	settings/credentials/*
Login options / Account alerts	Switches saved in SharedPreferences	ToggleSettingsActivity
Link accounts, Snapshot, Connected Apps, Help	Placeholders left as practice tasks	ComingSoonActivity

> **Try it** Demo sign on: **demouser** / **Finnova@2026**. Then tap **More → Profile & Settings**.

2. Open and run it in Android Studio

A. Get the code

1. Android Studio → **File → New → Project from Version Control…**
2. URL: `https://github.com/iEswar23/FinNova-Bank.git` → choose a folder → **Clone**.
3. When asked, click **Trust Project**. Gradle sync starts automatically (first time takes a few minutes).

B. Check the toolchain

- **Settings → Build, Execution, Deployment → Build Tools → Gradle → Gradle JDK**: use the bundled JDK 17 or 21.
- **Tools → SDK Manager**: Android 15 (API 35) SDK Platform installed. The project uses compileSdk 35, minSdk 26.
- If sync fails with *SDK location not found*, open **File → Project Structure → SDK Location** and pick your SDK; Android Studio writes `local.properties` for you (it is git-ignored).

C. Run

1. **Tools → Device Manager → Create Device** → Pixel 7 → API 34 or 35 image → Finish.
2. Select the **app** run configuration and the emulator in the toolbar → **Run ▶** (Shift+F10 / Ctrl+R on Mac).
3. Sign on, open Profile & Settings, edit a value, press Save, and watch it update on the list.

D. Run the tests

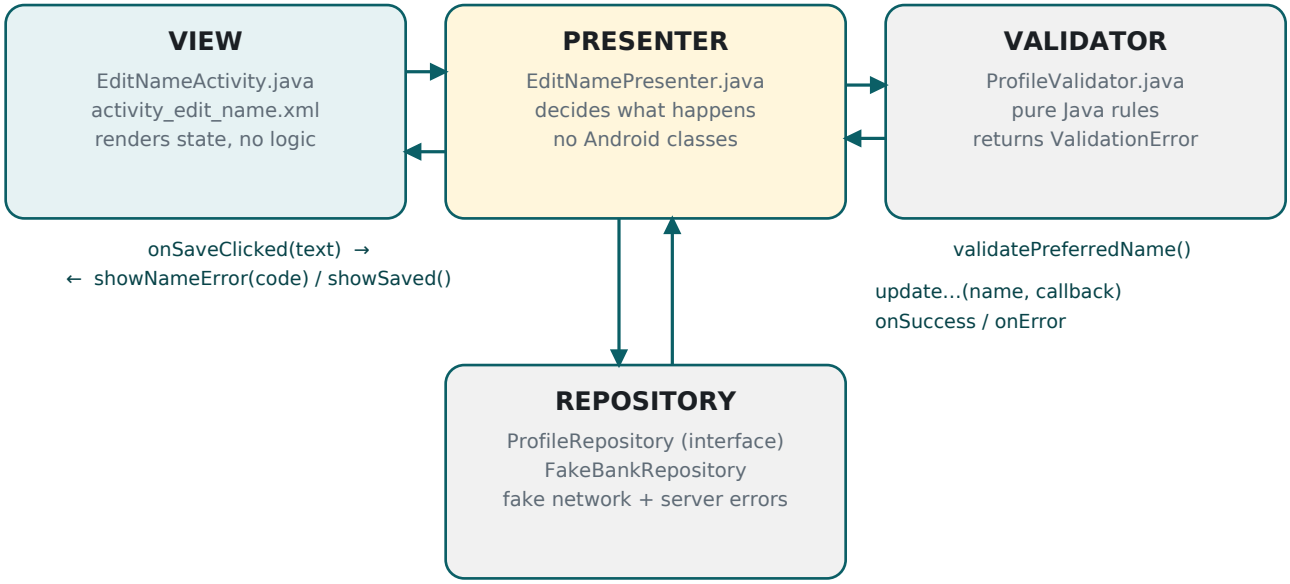
- Open `app/src/test/…/ProfileValidatorTest.java` and click the green ▶ next to the class name.
- Or in the **Terminal** tab: `./gradlew testDebugUnitTest` (Windows: `gradlew.bat testDebugUnitTest`). All **43** tests should pass.
- UI test (needs a running emulator): right-click `EditNameActivityTest` → Run.

Tech stack

Item	Version / choice	Why
Language	Java 17 (source/target)	Matches typical enterprise Android codebases
UI	XML layouts + ViewBinding + Material Components 1.12	Type-safe view access without findViewById
Architecture	MVP with Contract interfaces	Testable presenters, thin activities
Build	Android Gradle Plugin 8.7.3, Gradle 8.11.1 wrapper (Groovy DSL)	Stable, widely used
Testing	JUnit 4, Mockito 5, Espresso 3.6	Industry-standard unit + UI testing
Min / target SDK	26 / 35	Adaptive icons, modern APIs
Security basics	FLAG_SECURE in release, backups disabled, masked email/phone, password never trimmed	Bank-style defaults

3. MVP architecture

MVP splits every screen into three roles. The goal is simple: **all decisions live in the Presenter**, which is plain Java, so you can test them in milliseconds on your laptop without an emulator.



Layer	Responsibility	Must NOT do
View (Activity / Fragment + XML)	Inflate layout, forward clicks and text changes, render what the Presenter says (errors, loading, navigation)	Validate input, call repositories, hold business state
Presenter	Validate via ProfileValidator, call repository, map results/errors to View methods, track 'submitted' state	Import android.* (except annotations), reference Context, R.string or Views
Model (Repository + data classes)	Fetch/update data, return success or ServerException	Know about screens
Contract	Interface pair View + Presenter: the screen's API in one file	—

One full request, end to end (Save preferred name)

1. User taps **Save** → `EditNameActivity.save()` hides the keyboard and calls `presenter.onSaveClicked(text)`.
2. Presenter sets `submitted = true` and calls `validator.validatePreferredName(text, currentName)`.
3. Validator trims, then checks required → length → characters → same-as-current and returns the first failure as a `ValidationError` code (or VALID).
4. Presenter calls `view.showNameError(code)`. The Activity maps code → string with `ErrorMessages` and sets it on the `TextInputLayout`. If invalid, stop here.
5. If valid: `view.showLoading(true)` → `repository.updatePreferredFirstName(...)`.

6. Callback arrives ~0.7 s later. Presenter checks `getView() != null` (the user may have left), hides loading, then `showSaved()` (Toast + finish) or `showServerError()`.
7. Profile & Settings `onResume()` reloads the profile so the new name appears.

> **Tip** Why error *codes* instead of strings? Tests assert on stable codes, the validator stays pure Java, and copy changes only touch `strings.xml`. This is the pattern most bank apps use for server error codes too.

4. Project structure

Folder map

```
FinNova-Bank/
├── app/build.gradle                    dependencies, SDK levels, viewBinding
├── app/src/main/AndroidManifest.xml    activities, launcher = LoginActivity
├── app/src/main/java/com/eswar/finnova/
│   ├── base/        BaseView · BasePresenter<V> · BaseActivity (toolbar, back, FLAG_SECURE)
│   ├── data/model/  UserProfile · Address · Income · Account   (immutable, with…() copies)
│   ├── data/repository/
│   │                ProfileRepository · AuthRepository · AccountRepository (interfaces)
│   │                FakeBankRepository (in-memory backend) · DataCallback · ServerException
│   ├── di/          Injection  (manual dependency injection / service locator)
│   ├── validation/  ProfileValidator · ValidationResult · ValidationError
│   ├── util/        Formatters (mask/format) · TaskRunner · MainThreadTaskRunner
│   └── ui/
│       ├── login/            LoginContract · LoginPresenter · LoginActivity
│       ├── main/             MainActivity (bottom navigation)
│       ├── accounts/         AccountsContract · AccountsPresenter · AccountsFragment · AccountsAdapter
│       ├── benefits/         InfoFragment (Benefits + Offers)
│       ├── more/             MoreFragment
│       ├── common/           ErrorMessages · SimpleTextWatcher · UiUtils
│       ├── profile/settings/ ProfileRow · ProfileSettingsContract/Presenter/Activity
│       ├── profile/name/     EditNameContract · EditNamePresenter · EditNameActivity
│       ├── profile/email/    …same three files
│       ├── profile/phone/    …same three files
│       ├── profile/address/  AddressForm + Contract · Presenter · Activity
│       ├── profile/income/   IncomeForm + Contract · Presenter · Activity
│       └── settings/         credentials/* · comingsoon/* · ToggleSettingsActivity
├── app/src/main/res/
│   ├── layout/      activity_*.xml · fragment_*.xml · view_settings_row.xml · view_toolbar.xml
│   ├── values/      strings.xml (all copy + error messages) · colors · themes · arrays · dimens
│   └── drawable/    vector icons · launcher icon
├── app/src/test/        ProfileValidatorTest · FormattersTest · 4 presenter tests (JUnit + Mockito)
└── app/src/androidTest/ EditNameActivityTest (Espresso)
```

Naming conventions used everywhere

Thing	Pattern	Example
Screen package	ui/<area>/<screen>/	ui/profile/email/
Contract	<Screen>Contract with nested View and Presenter	EditEmailContract.View
Layout	activity_<screen>.xml / fragment_<screen>.xml	activity_edit_email.xml
View ids	<field>Layout (TextInputLayout) + <field>Input (EditText)	emailLayout, emailInput
Error codes	<FIELD>_<PROBLEM>	EMAIL_INVALID_FORMAT
Error strings	err_<field>_<problem>	err_email_invalid
Tests	<Class>Test, method = scenario_expectedResult	save_withInvalidName_showsError_andDoesNotCallServer

5. Profile validation rules

All rules live in `validation/ProfileValidator.java`. Four principles apply to every field:

1. **Trim first** (except passwords, where spaces are part of what the user typed).
2. **Check in a fixed order**: required → length → format/characters → business rule.
3. **Return only the first failure**, so the user sees one clear message per field.
4. **Client validation is for UX only.** The server validates again (see the fake server errors).

Field	Rule (in order)	Error code
Preferred first name	Required	NAME_REQUIRED
	≤ 30 characters (XML also limits with maxLength + counter)	NAME_TOO_LONG
	Letters; single space, hyphen or apostrophe only *between* letters	NAME_INVALID_CHARACTERS
	Different from current name (case-insensitive)	NAME_SAME_AS_CURRENT
Email	Required · ≤ 254 chars · name@domain.tld, no spaces, no '..'	EMAIL_REQUIRED / _TOO_LONG / _INVALID_FORMAT
	Not the current email · confirm field required and must match	EMAIL_SAME_AS_CURRENT / _CONFIRM_REQUIRED / _MISMATCH
	Server: email used by another customer	ServerException EMAIL_IN_USE
Mobile phone	Strip non-digits, drop leading 1 → must be 10 digits	PHONE_REQUIRED / PHONE_INVALID_LENGTH
	Area code and exchange can't start with 0 or 1 · not current number	PHONE_INVALID_AREA_CODE / _EXCHANGE / _SAME_AS_CURRENT
Street address	Required · 5–60 chars · letters, digits, space . , ' # / -	ADDRESS_LINE1_REQUIRED / _TOO_SHORT / ADDRESS_TOO_LONG / _INVALID_CHARACTERS
	No P.O. Box (residential address required)	ADDRESS_PO_BOX_NOT_ALLOWED
Apt / suite	Optional; if present same character and P.O. box rules	same as above

Field	Rule (in order)	Error code
City	Required · letters with single space . ' - · ≤ 40	CITY_REQUIRED / CITY_INVALID
State	Required · one of 50 states + DC + PR	STATE_REQUIRED / STATE_INVALID
ZIP	Required · 12345 or 12345-6789 · not 00000	ZIP_REQUIRED / ZIP_INVALID
Employment	Required (dropdown)	EMPLOYMENT_STATUS_REQUIRED
Annual income	Required · whole dollars, optional commas · 0 to 9,999,999	INCOME_REQUIRED / _INVALID / _OUT_OF_RANGE
Monthly housing	Required · 0 to 99,999 · 12 × monthly ≤ annual income	HOUSING_REQUIRED / _INVALID / _OUT_OF_RANGE / _EXCEEDS_INCOME
User ID	Required · 6–30 · starts with a letter · letters, digits . _	USER_ID_REQUIRED / _LENGTH / _INVALID
New password	Required · 8–32 · no spaces · upper · lower · digit · special	PASSWORD_REQUIRED / _LENGTH / _HAS_SPACE / _MISSING_*
	Must not contain User ID · must differ from current · confirm must match	PASSWORD_CONTAINS_USER_ID / _SAME_AS_CURRENT / _MISMATCH

When errors appear (validation UX)

- **Before the first Save**: typing never shows errors. Nobody likes red text while they're still typing.
- **On Save**: every field is validated, all errors are shown, and (on Address) focus jumps to the first invalid field.
- **After a failed Save**: fields re-validate live on every keystroke, so errors disappear the moment they're fixed.
- **While saving**: progress bar visible, Save disabled (prevents double submits).
- **Server errors** that belong to a field (email in use, wrong current password, User ID taken) are shown *on that field*; anything else appears in a dialog.

6. Task 1 — Profile page validation, step by step

This is how to approach a validation story on a real team, using a concrete example you can do in this repo right now. The rule is not implemented yet, so you'll write it yourself.

> **Requirement Story FIN-101:** As a customer, I can't save a preferred first name shorter than 2 letters, so greetings never look like “Hello, A”.
> **Acceptance criteria:** (1) “A” or “ a ” shows *“Name must be at least 2 letters.”* under the field on Save. (2) “Al” saves. (3) The error clears as soon as the user types a valid name. (4) Unit tests cover the rule.

STEP 1 Create a branch

Android Studio: bottom-right branch widget → **New Branch…** → `feature/FIN-101-name-min-length`. Or in the Terminal tab:

```
git checkout main
git pull
git checkout -b feature/FIN-101-name-min-length
```

STEP 2 Find the code for the screen

- Press **Shift twice** (Search Everywhere) and type *Preferred first name*. You land in `strings.xml` → `row_preferred_name`.
- Right-click the string name → **Find Usages** (Alt+F7) → it leads to `EditNameActivity`.
- On a running app, **Tools → Layout Inspector** shows which layout and view id is on screen. Great for unfamiliar codebases.
- Read the **Contract** first (`EditNameContract`): it lists everything the screen can do.

STEP 3 Write the failing test first (TDD)

Open `ProfileValidatorTest` and add:

app/src/test/.../validation/ProfileValidatorTest.java

```
@Test public void name_tooShort() {
    assertError(ValidationError.NAME_TOO_SHORT, validator.validatePreferredName("A", "Alex"));
    assertError(ValidationError.NAME_TOO_SHORT, validator.validatePreferredName(" a ", "Alex"));
    assertValid(validator.validatePreferredName("Al", "Alex"));
}
```

It won't compile yet: `NAME_TOO_SHORT` doesn't exist. That's expected; the test describes the goal.

STEP 4 Add the error code

validation/ValidationError.java

```
public enum ValidationError {
    // Preferred first name
    NAME_REQUIRED,
    NAME_TOO_SHORT,          // <-- new
    NAME_TOO_LONG,
    ...
```

STEP 5 Implement the rule in the validator

Add a constant and put the check right after “required” (order matters: an empty field should say *required*, not *too short*).

validation/ProfileValidator.java

```
public static final int NAME_MIN = 2;
public static final int NAME_MAX = 30;

public ValidationResult validatePreferredName(@Nullable String input, @Nullable String current) {
    String name = trim(input);
    if (name.isEmpty()) return invalid(ValidationError.NAME_REQUIRED);
    if (name.length() < NAME_MIN) return invalid(ValidationError.NAME_TOO_SHORT);   // <-- new
    if (name.length() > NAME_MAX) return invalid(ValidationError.NAME_TOO_LONG);
    if (!NAME.matcher(name).matches()) return invalid(ValidationError.NAME_INVALID_CHARACTERS);
    if (name.equalsIgnoreCase(trim(current))) return invalid(ValidationError.NAME_SAME_AS_CURRENT);
    return ok();
}
```

Run the test (green ▶ in the gutter). It should pass now.

STEP 6 Add the message and map it

res/values/strings.xml

```
<string name="err_name_too_short">Name must be at least 2 letters.</string>
```

ui/common/ErrorMessages.java

```
case NAME_REQUIRED: return R.string.err_name_required;
case NAME_TOO_SHORT: return R.string.err_name_too_short;     // <-- new
case NAME_TOO_LONG: return R.string.err_name_too_long;
```

> **Tip** Forget this mapping and the switch falls to *default* and shows “Something went wrong”. A good extra unit test: loop over `ValidationError.values()` and assert none maps to `something_went_wrong`.

STEP 7 Presenter: nothing to change (and why)

`EditNamePresenter` already calls the validator on Save and on every keystroke after the first Save, then passes the code to `view.showNameError()`. Because the rule lives in one place, the new check works on this screen automatically. Add one presenter test to prove acceptance criteria 1 and 3:

app/src/test/.../ui/profile/EditNamePresenterTest.java

```
@Test
public void save_withOneLetter_showsTooShort_thenClearsWhenFixed() {
    presenter.onSaveClicked("A");
    presenter.onNameChanged("Al");

    InOrder order = inOrder(view);
    order.verify(view).showNameError(ValidationError.NAME_TOO_SHORT);
    order.verify(view).showNameError(null);
    verify(view, never()).showSaved();
}
```

STEP 8 View and XML check

- `activity_edit_name.xml`: the field is a `TextInputLayout` (error text space, counter 30) with a `TextInputEditText` using `inputType="textPersonName|textCapWords"`, `maxLength="30"` and `imeOptions="actionDone"` (keyboard Done = Save).
- `EditNameActivity.showNameError()` → `ErrorMessages.apply(binding.nameLayout, error)`. Passing `null` clears the error.
- Optional polish: update `preferred_name_help` to mention the 2-letter minimum.

STEP 9 Run all unit tests

```
./gradlew testDebugUnitTest
# report: app/build/reports/tests/testDebugUnitTest/index.html
```

In Android Studio: right-click the `test` folder → **Run 'Tests in …' with Coverage** to see which lines of `ProfileValidator` are covered.

STEP 10 Manual test on the emulator

#	Input	Action	Expected
1	(empty)	Save	“Enter your preferred first name.”
2	A	Save	“Name must be at least 2 letters.”
3	A → Al	Type after failed Save	Error disappears while typing
4	Al	Save	Toast “Your changes were saved.”, back on list, subtitle shows Al
5	Alex1	Save	Letters-only message
6	Sai Ram (two spaces)	Save	Letters-only message
7	Eswar	Save	Saves as “Eswar” (trimmed)
8	alex (current is Alex)	Save	“This is already your preferred first name.”
9	31+ chars pasted	Paste	Field stops at 30 (maxLength); counter shows 30/30
10	Valid name	Rotate device during save	No crash (presenter checks getView() != null)
11	Any error	TalkBack on	Error text is announced
12	Any	Settings → Display size / Font size max	Error text wraps, nothing clipped

STEP 11 Debug when something is off

- Click in the gutter of `EditNamePresenter.onSaveClicked` to set a breakpoint → run with **Debug** (Shift+F9) → press Save → step with F8, inspect `result` in the Variables pane.
- **Logcat**: filter `package:mine level:error` to see crashes only.
- Error not showing? Check the View method is actually called (breakpoint) and that the layout id is the `TextInputLayout`, not the EditText.

STEP 12 Commit, push and open a pull request

```
git add -A
git commit -m "FIN-101: Require at least 2 letters for preferred first name"
git push -u origin feature/FIN-101-name-min-length
# then on GitHub: Compare & pull request -> describe change + paste test results
```

Self-review checklist before the PR

- Rule is in **ProfileValidator** only, not duplicated in the Activity.
- New code has a **string resource** and an **ErrorMessages** mapping.
- Unit tests for valid, invalid and boundary values (1, 2, 30, 31 characters).
- No hard-coded text in Java or XML; no `Log.d` of personal data (names, emails, phone numbers).
- All tests green; app manually tested on the emulator.

7. More practice on the same screens

Each exercise follows the same 12 steps. They're ordered from easiest to hardest.

#	Exercise	Files you'll touch	Skill practised
1	Name: block three repeated letters in a row (“Aaaalex”)	ProfileValidator, enum, strings, tests	Regex
2	Email: lowercase before saving and comparing	EditEmailPresenter, test	Normalisation
3	Phone: reject 555-01xx fictional numbers	ProfileValidator, test	Business rules
4	Address: Save button disabled until required fields are non-empty	Contract (setSaveEnabled), Presenter, Activity, test	Reactive UI state
5	Income: show live “$95,000” formatting while typing	TextWatcher in View, parseMoney already handles commas	Input formatting
6	Show a “Discard changes?” dialog when pressing back with edits	Presenter tracks dirty state; View shows dialog	Lifecycle + back handling
7	Keep typed values and errors after rotation	onSaveInstanceState, Presenter state restore	Config changes
8	Build Connected Apps screen (list + revoke) with MVP	New Contract/Presenter/Activity/layout, repository method	Full feature end to end
9	Replace FakeBankRepository with Retrofit + MockWebServer	New RetrofitProfileRepository, Injection	Networking
10	Replace Thread.sleep in the Espresso test with an IdlingResource	androidTest + repository hook	Reliable UI tests

Common build problems

Symptom	Fix
Cannot resolve symbol ActivityEditNameBinding	Build → Rebuild Project. ViewBinding classes are generated from XML; a broken XML file stops generation.
Unresolved R / R.string.xxx	Check the string exists in values/strings.xml and the import is com.eswar.finnova.R (not android.R).
SDK location not found	File → Project Structure → SDK Location, or create local.properties with sdk.dir=…
Gradle JDK mismatch	Settings → Gradle → Gradle JDK → jbr-17 or jbr-21.
App installs but crashes on launch	Logcat → filter level:error → read the first 'Caused by' line. Usually a missing manifest entry or wrong view id.
Test passes alone but fails in suite	Shared state. Create a new FakeBankRepository in @Before (as the tests already do).

Interview-ready answers

Question	Short answer
Why MVP over putting logic in the Activity?	Presenter is plain Java → fast JVM unit tests, smaller Activities, logic survives UI rewrites.
How do you avoid memory leaks in MVP?	attachView/detachView with lifecycle; null-check getView() in async callbacks; never store Context in the Presenter.
MVP vs MVVM?	MVP: Presenter calls View methods through an interface. MVVM: View observes LiveData/StateFlow from a ViewModel that survives rotation.
Where should validation live?	In a reusable validator called by the Presenter; the server validates again. The View only displays results.
How do you show server errors?	Map error codes to field errors when they belong to a field, otherwise a dialog; never show raw stack traces.
How do you protect PII on screen?	Mask email/phone, FLAG_SECURE in release, no PII in logs, disable backups.

> **Note** Keep employer code separate: build these practice features only in this personal repo, and never copy code, screenshots, API names or data from work projects into it.